

* Associative Array (map, dictionary) is a collection of (key, value) pairs, such that each possible key appears at most once in the collection.

e.g. a collection of students

each pair (student) : (ID, info)

An associative array is an abstract data type (ADT).

Which data structure should we use to maintain an AA?

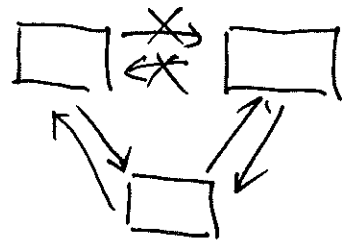
What operations are associated with this ADT?

1. insert a pair to the collection
2. delete a pair from the collection
3. modify a pair in the collection
4. lookup a value of a given key

Two major data structures: Search Tree
and
Hash table

Suppose we use list to maintain an associative array.

Time complexity of insertion: $O(1)$



lookup: $O(n)$ for aver
&
worst

10 20 30 40 50

best case : lookup (10) ... $O(1)$

worst case : lookup (50) ... $O(n)$

average ... $\frac{n}{2} = O(?)$

delete: we have to lookup the
node first, then delete.

So, aver & worst $O(n)$

`std::set` & `std::map` are implemented using Search Tree.

sets are containers that store (only) unique keys.

- * elements in associative containers are referenced by their keys , NOT absolute positions.
- * Set: the value of an element is the key to it.
- * No two elements in the container can have the same key.
- * All elements are ordered (by a strict order).

e.g. `int myint[] = { 75, 23, 65, 42, 13 };`

```
std::set<int> myset (myint, myint+5);
```

myset:

	13	23	42	65	75	
	↑					↑
	begin	++	++	++	++	end

- * set supports bidirectional iterators.

member functions:

begin() end() empty() size()

find()

find(value) returns an iterator pointing to value

insert()

insert(value) if value exists, do nothing;
otherwise, insert value

return a pair <iterator, bool>

iterator is either pointing to the newly inserted element
or the element with value

bool: true — new element is inserted
false — value exists

erase()

erase(iterator) or erase(iterator first, iterator last)

erase(value): returns the number of elements being
erased. (0 or 1)